Void Companion: Binary Distinction and the Four-Vertex Closure

Johannes Michael Wielsch
Companion note to *Void and Form*
Project DOI: 10.5281/zenodo.19491035

May 2026

Abstract

This companion studies a narrow licensing problem. A formal system may say that there are two positions. What data internal to the system license that count as two, rather than one position named twice or a larger carrier with unmentioned points?

The entry pressure is the Leibnizian principle usually called the identity of indiscernibles: if no expressible difference separates two alleged positions, the system has no internal ground for holding them apart. This principle is not itself proved by Agda. It motivates the formal threshold at which the question becomes checkable.

In intensional Martin-Löf type theory, formalized in Agda under `--safe --without-K`, the threshold is represented by a Distinction record. The record contains a carrier S, two distinguished positions left and right, a separation witness separated : ¬ (left ≡ right), and a cover cover showing that every carrier element is one of those two positions. Thus separated is anti-collapse data, and cover is anti-surplus data.

From that record the checked core proves a conditional chain. Every Distinction is boundary-preservingly isomorphic to the fresh two-point normal form Two. The endomorphism space Two → Two has exactly four cases: the two constants, identity, and swap. Under the stated representation contract—separation of cases, complete realisation, and no surplus—the corresponding simple graph closure is K_4. The closure is then read back to the originating Distinction datum.

The result is not a theorem that all formal systems begin this way. It is a mechanically checked conditional result inside the stated Agda fragment. The broader theory of formulability is the architectural reading of this chain, not an additional Agda theorem.

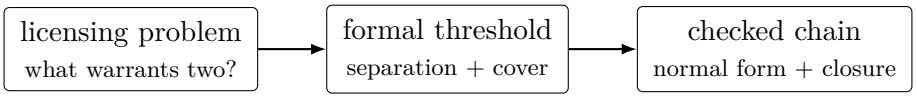
Reading key

The note begins with a small question and keeps returning to it: *a formal system says that there are two; what inside the system licenses that claim?* The answer is not sought in K_4, in a truth-value type, or in a later interpretation. The answer begins with the data needed to prevent collapse and hidden surplus.

The pre-formal part introduces the collapse pressure. The identity of indiscernibles says, in the form needed here, that two alleged positions cannot be maintained as two if the system can express no difference between them. This is a motivation for the formal datum, not a checked term of the development.

The formal threshold is crossed only when explicit data are supplied: carrier, left boundary, right boundary, separation, and cover. From that point onward the claims are local and conditional. Agda checks what follows from the record; it does not manufacture the pre-formal reason for asking for the record.

The checked part then follows one route: Distinction to Two, Two to its four endomorphism cases, the four cases to K_4 under the representation contract, and the closure back to the originating Distinction datum. Later interpretation is kept separate from this kernel.



Argument map

Licensing problem. The paper starts from the claim of two, not from a completed two-element object. The question is what internal data make such a claim stable.

Collapse pressure. The identity of indiscernibles supplies the pressure: without an expressible distinction, there is no internal warrant for maintaining two alleged positions as two.

Formal threshold. The Distinction record supplies the data used in this note: a carrier, two distinguished positions, a non-coincidence witness, and an exhaustive cover.

Checked core. Given the record, the code checks normal form, four-case endomorphism classification, K_4 closure under the stated representation conditions, and return to the originating datum.

Later layers. *Form* and the larger `Void.lagda.tex` development are later continuations. They are not premises of the kernel and do not change the status of the conditional theorem proved here.

Part I. The pre-formal threshold

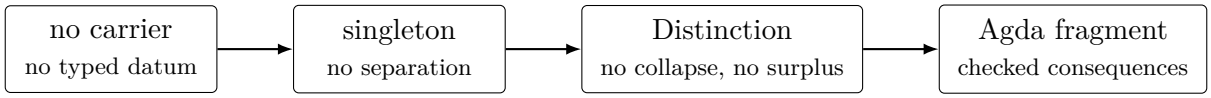
The licensing problem

The pre-formal part has one role: to explain why the formal development does not begin with an unexamined two-element type. A relation has two positions. A morphism has a source and a target. A judgement relates a term and a type. An equation has two sides, even when it identifies them. Each form already uses separability.

The licensing question asks what a system must have in order to keep two positions apart before further structure is read into them. Naming two points is not enough. If the names can collapse, the datum is not two. If further points can inhabit the carrier unaccounted, the datum is not exhaustively two.

The following display is not Agda code and not a theorem of MLTT. It is a schematic boundary marker for the passage into the checked part. The arrows are meta-level dependencies, not implication terms.

$$
\begin{array}{ll}
\text{Level 0 :} & \mathcal{V} \notin Set_\omega, \qquad \neg\exists x\,(x : \mathcal{V}). \\
\text{Level 1 :} & \mathrm{Sing}(S) := \exists p : S\,\forall x : S\,(x = p). \\
\text{Collapse :} & \mathrm{Sing}(S) \Longrightarrow \neg\exists a, b : S\,(a \neq b). \\
\text{Level 2 :} & \mathrm{Dist}(S) := \exists a, b : S\,(a \neq b \ \wedge\ \forall x : S\,(x = a \vee x = b)). \\
\text{Formal threshold :} & \mathrm{Dist}(S) \Longrightarrow \ S \simeq \mathbf{2}, \\
 & \qquad\qquad \mathrm{End}(\mathbf{2}) = \{c_L, c_R, \mathrm{id}, \sigma\}, \\
 & \qquad\qquad \text{representation closure} = K_4, \\
 & \qquad\qquad K_4 \Longrightarrow \mathrm{Dist}(S).
\end{array}
$$



Level 0 is not a hidden type. It marks the absence of a domain in which a term could already be typed. Level 1 shows why mere inhabitance is still collapse: if all elements are identified with one point, no internal separation is available. Level 2 is the first formal datum used by the note: two distinguished positions, a separation witness, and a cover saying that every carrier element is one of those positions.

Only the dependency from $\mathrm{Dist}(S)$ onward enters Agda. From that point, the schematic notation is replaced by checked terms.

Why two cannot occur without separation

Before any cardinality is chosen, a sharper question arises. Why can a system say that two positions are present when no structure available inside the system distinguishes them?

The relevant pressure is the identity of indiscernibles, used here in a modest form: a system cannot maintain a two-count by merely naming two positions if it has no expressible difference that keeps them apart. This is the reason the formal datum must include anti-collapse data.

This is why separation is not decoration. Without a witness that left and right do not coincide, the named positions can collapse. This is also why cover is not decoration. Two named positions do not by themselves rule out a third, unaccounted point in the carrier. A stable two-point datum needs both clauses: separation against collapse, and exhaustion against hidden surplus.

Once a carrier has been chosen, MLTT/Agda gives a concrete execution of this principle. The term displayed later as indiscernibility-implies-equality is one realization of the principle, not its source.

Why two named points are not enough

The formal answer cannot be merely "choose two elements". In MLTT, a carrier may contain left and right and still fail to be a two-point structure. It can fail in two ways.

First, the two displayed points may coincide. The field separated prevents this collapse by requiring a witness of $\neg$ (left $\equiv$ right). Second, the carrier may contain additional points. The field cover prevents this surplus by requiring every element to be equal to left or to right. Thus separated is anti-collapse data, and cover is exhaustion data.

Why binary, not unary or ternary

The binary datum is introduced before any truth-value reading. At the pre-formal threshold, the datum must answer two local demands: it must not collapse to one, and it must not hide further positions. This argument is conceptual, not an Agda theorem; the formal results below do not depend on it. It is recorded here so that the antecedent of the machine-checked conditional is not opaque.

Unary collapses. A single occurrence carries no separation. The schematic predicate $\mathrm{Sing}(S)$ records exactly this: every element equals the chosen point, and no pair (a, b) with $a \neq b$ can be exhibited. Whatever appears within S is the same appearance. Formalisation cannot start here, because relations, morphisms, judgements, and equations all require two positions before they can be written down.

Ternary carries binary data plus surplus. Three positions a, b, c require three pairwise separations $a \neq b$, $b \neq c$, $a \neq c$. Each pairwise separation has binary form. A ternary record therefore does not remove the binary problem; it adds labels and a cover over a larger family of positions. Within the record format studied here, the ternary case is analysed through its pairwise separations plus the additional data needed to account for the third label.

Binary supplies the minimal datum. A binary distinction supplies exactly one separation $a \neq b$ together with the exhaustion clause that nothing further is present. No additional structure (order, incidence, selection) is assumed; any such structure that arises later is obtained from the four endomorphism cases of the normal form, not added to the datum. This is why the formal development below begins with Distinction and not with a richer or poorer record.

These observations are later reflected inside the formal development. A singleton carrier admits no inhabitant of Distinction; at $n = 2$ the n-ary version of the record is mutually convertible with Distinction; and higher arities carry pairwise binary separation witnesses together with a nested binary cover.

Part II. The formal threshold

1 Setting

Part I fixed the question that Agda is asked to answer. The formal development now begins after the threshold has been specified. The setting is ordinary intensional Martin-Löf type theory as implemented in Agda. The code snippets below use the standard library for equality, sums, products, the empty type, and negation. This companion records the minimal spine of the argument; the larger `Void.lagda.tex` development rebuilds more infrastructure internally.

The fresh type Two has the same shape as Bool, but it is introduced without the truth-value reading. In this note, Two is used as the normal form of the Distinction record, not as a pre-given truth-value object.

```
-- § Safe fragment: no postulates and no Axiom K.
{-# OPTIONS --safe --without-K #-}

-- § Self-contained kernel companion module.
module VoidCompanion where

-- § Standard infrastructure: equality, absurdity, sums, pairs, and negation.
open import Relation.Binary.PropositionalEquality using (_≡_; refl; sym; trans; cong)
open import Data.Empty using (⊥; ⊥-elim)
open import Data.Sum using (_⊎_; inj₁; inj₂)
```

```
open import Data.Product using (Σ; _,_)
open import Relation.Nullary using (¬_)
```

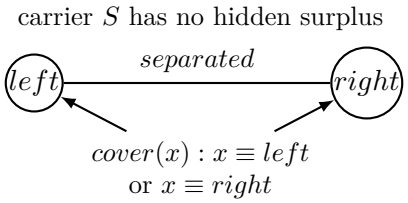
The two-point normal form is the following type.

```
-- § The two-point normal form has exactly two constructors.
data Two : Set where
 L R : Two

-- § The two boundary names of Two do not collapse.
L≠R : ¬ (L ≡ R)
L≠R ()

-- § The opposite inequality is obtained by symmetry of equality.
R≠L : ¬ (R ≡ L)
R≠L p = L≠R (sym p)
```

2 Distinction

A binary distinction is not just a type with two named points. The record supplies the explicit data that answer the threshold question: a carrier, two distinguished positions, a proof that those positions do not coincide, and a cover showing that every carrier element is one of them. The separated field blocks collapse by indiscernibility of the two distinguished positions; the cover field blocks hidden surplus in the carrier. Together they are the exact hypothesis that makes the normal-form theorem true.

```
-- § Binary distinction: carrier, left/right boundaries, separation, coverage.
record Distinction : Set₁ where
 field
  S         : Set
  left      : S
  right     : S
  separated : ¬ (left ≡ right)
  cover     : (x : S) → (x ≡ left) ⊎ (x ≡ right)
```



The canonical inhabitant is obtained from Two itself.

```
-- § The canonical cover of Two is exhaustive by case analysis.
Two-cover : (x : Two) → (x ≡ L) ⊎ (x ≡ R)
Two-cover L = inj₁ refl
Two-cover R = inj₂ refl

-- § The canonical two-point distinction.
Two-distinction : Distinction
Two-distinction = record
 { S         = Two
 ; left      = L
 ; right     = R
 ; separated = L≠R
 ; cover     = Two-cover
 }
```

3 Cardinality of distinction

The earlier prose in Part I argued that one occurrence collapses, ternary records retain binary separations plus additional data, and the binary case supplies the minimal candidate. The argument was explicitly marked as conceptual. At this point, after the Distinction record is in place, the same content can be stated in the precise form available to this framework. A singleton carrier admits no inhabitant of Distinction at all, and the n-ary generalisation of the record is, at $n = 2$, mutually convertible with the binary case at the displayed carrier and boundary fields. Higher arities carry pairwise binary separation witnesses plus an exhaustive cover that is structurally a nested binary disjunction. The result is local to this record-based `--safe --without-K` fragment.

```
-- § Finite arities and finite positions.
open import Data.Nat using (ℕ; zero; suc)
open import Data.Fin using (Fin) renaming (zero to fz; suc to fs)

-- § N-ary distinction template: labelled points, pairwise separation, coverage.
record Distinction-n (n : ℕ) : Set₁ where
 field
  S       : Set
  point   : Fin n → S
  distinct : (i j : Fin n) → ¬ (i ≡ j) → ¬ (point i ≡ point j)
  cover : (x : S) → Σ (Fin n) (λ i → x ≡ point i)
```

Unary collapses formally. No singleton carrier hosts a Distinction: if every two elements of the carrier are equal, the boundary points are equal and separated is falsified.

```
-- § A carrier whose elements all coincide cannot host a binary distinction.
singleton-no-distinction :
 (d : Distinction) →
 ((x y : Distinction.S d) → x ≡ y) →
 ⊥
singleton-no-distinction d hyp =
 Distinction.separated d (hyp (Distinction.left d) (Distinction.right d))
```

The same content holds for Distinction-n at $n = 1$: any two elements of its carrier are identified, so no separation between distinct positions can be inhabited.

```
-- § A unary n-ary distinction carrier collapses every two elements.
distinction-n-1-collapses :
 (d : Distinction-n 1) →
 (x y : Distinction-n.S d) → x ≡ y
distinction-n-1-collapses d x y
 with Distinction-n.cover d x | Distinction-n.cover d y
... | fz , px | fz , py = trans px (sym py)
... | fz , _ | fs () , _
... | fs () , _ | _
```

The collapse on Distinction-n 1 is one half of the "unary kills distinction" content; on its own, it identifies every two elements of the carrier to coincide. The bridge to the original record is the next step: any Distinction that injects into such a carrier is immediately impossible, because the two separated boundary points become equal under the injection and separated is contradicted. This makes the joint statement–a unary carrier hosts no Distinction–a single checked term and not a verbal combination of two lemmas.

```
-- § No binary distinction injects into a unary n-ary carrier.
no-distinction-on-Dn1 :
 (d1 : Distinction-n 1) →
 (d : Distinction) →
 (φ : Distinction.S d → Distinction-n.S d1) →
 ((x y : Distinction.S d) → φ x ≡ φ y → x ≡ y) →
 ⊥
```

```
no-distinction-on-Dn1 d1 d φ φ-inj =
 Distinction.separated d
  (φ-inj (Distinction.left d) (Distinction.right d)
    (distinction-n-1-collapses d1
     (φ (Distinction.left d))
     (φ (Distinction.right d))))
```

Binary equals the existing record. At $n = 2$ the n-ary record is mutually convertible with the original Distinction. The two constructions are mechanical and demonstrate that no boundary information is lost in either direction. Because the records contain function fields, the companion does not claim an unqualified record equality of the conversions; instead it states the recoverable carrier and boundary fields explicitly.

```
-- § Convert a binary distinction into the n-ary template at n=2.
distinction-to-Dn2 : Distinction → Distinction-n 2
distinction-to-Dn2 d = record
 { S       = Distinction.S d
 ; point   = pt
 ; distinct = dist
 ; cover = cov
 }
 where
  open Distinction d
  pt : Fin 2 → S
  pt fz = left
  pt (fs fz) = right

  dist : (i j : Fin 2) → ¬ (i ≡ j) → ¬ (pt i ≡ pt j)
  dist fz fz       h _ = h refl
  dist fz (fs fz) _ p = separated p
  dist (fs fz) fz _ p = separated (sym p)
  dist (fs fz) (fs fz) h _ = h refl

  cov : (x : S) → Σ (Fin 2) (λ i → x ≡ pt i)
  cov x with cover x
  ... | inj₁ p = fz , p
  ... | inj₂ p = fs fz , p

 -- § Convert the n=2 template back into the binary distinction record.
Dn2-to-distinction : Distinction-n 2 → Distinction
Dn2-to-distinction d = record
 { S       = Distinction-n.S d
 ; left    = Distinction-n.point d fz
 ; right   = Distinction-n.point d (fs fz)
 ; separated = Distinction-n.distinct d fz (fs fz) λ ()
 ; cover = cov
 }
 where
  open Distinction-n d
  cov : (x : S) → (x ≡ point fz) ⊎ (x ≡ point (fs fz))
  cov x with cover x
  ... | fz , p      = inj₁ p
  ... | fs fz , p = inj₂ p

-- § Recoverable carrier and boundary fields for the n=2 roundtrips.
record DistinctionDn2Equivalence : Set₁ where
 field
  from-to-carrier : (d : Distinction) →
   Distinction.S (Dn2-to-distinction (distinction-to-Dn2 d)) ≡ Distinction.S d
  from-to-left    : (d : Distinction) →
   Distinction.left (Dn2-to-distinction (distinction-to-Dn2 d)) ≡ Distinction.left d
  from-to-right   : (d : Distinction) →
   Distinction.right (Dn2-to-distinction (distinction-to-Dn2 d)) ≡ Distinction.right d
  to-from-carrier : (d : Distinction-n 2) →
   Distinction-n.S (distinction-to-Dn2 (Dn2-to-distinction d)) ≡ Distinction-n.S d
  to-from-point-0 : (d : Distinction-n 2) →
   Distinction-n.point (distinction-to-Dn2 (Dn2-to-distinction d)) fz ≡ Distinction-n.point d fz
  to-from-point-1 : (d : Distinction-n 2) →
```

```
  Distinction-n.point (distinction-to-Dn2 (Dn2-to-distinction d)) (fs fz) ≡ Distinction-n.point d (fs fz)

 -- § The n=2 conversions preserve the displayed carrier and boundary fields.
theorem-distinction-Dn2-equivalence : DistinctionDn2Equivalence
theorem-distinction-Dn2-equivalence = record
 { from-to-carrier = λ _ → refl
 ; from-to-left    = λ _ → refl
 ; from-to-right = λ _ → refl
 ; to-from-carrier = λ _ → refl
 ; to-from-point-0 = λ _ → refl
 ; to-from-point-1 = λ _ → refl
 }
```

Higher arity carries only pairwise binary plus a nested disjunction. At any n, the n-ary distinction record carries a binary separation witness for every ordered pair of distinct positions. This witness is the distinct field itself; no further structure is needed to extract it.

```
-- § Every n-ary distinction supplies binary separation for every distinct pair.
pairwise-binary :
 {n : ℕ} (d : Distinction-n n) →
 (i j : Fin n) → ¬ (i ≡ j) →
 ¬ (Distinction-n.point d i ≡ Distinction-n.point d j)
pairwise-binary = Distinction-n.distinct
```

At $n = 3$, the exhaustive cover of three positions is structurally the nested binary disjunction $A_0 \uplus (A_1 \uplus A_2)$, where A_i records that an element equals position i. No primitive ternary disjunction is invoked; the three-case split is two binary splits.

```
-- § The three-point cover is a nested binary disjunction.
cover-3-nested :
 (d : Distinction-n 3) (x : Distinction-n.S d) →
 (x ≡ Distinction-n.point d fz)
  ⊎ ((x ≡ Distinction-n.point d (fs fz))
     ⊎ (x ≡ Distinction-n.point d (fs (fs fz))))
cover-3-nested d x with Distinction-n.cover d x
... | fz , p          = inj₁ p
... | fs fz , p       = inj₂ (inj₁ p)
... | fs (fs fz) , p = inj₂ (inj₂ p)
```

The two preceding lemmas show only that a ternary record *carries* binary content. The stronger claim, that the ternary record is *nothing more* than this binary content, requires the converse direction: any package of three pairwise separations together with a binary-nested cover must reconstruct a Distinction-n 3. The following record names exactly the binary data extractable from a ternary distinction, and the two conversions below show the reduction in both directions.

```
-- § Binary data package sufficient to reconstruct a ternary distinction.
record Dn3-binary-data : Set₁ where
 field
  S   : Set
  p0  : S
  p1  : S
  p2  : S
  sep01 : ¬ (p0 ≡ p1)
  sep02 : ¬ (p0 ≡ p2)
  sep12 : ¬ (p1 ≡ p2)
  cover : (x : S) → (x ≡ p0) ⊎ ((x ≡ p1) ⊎ (x ≡ p2))

 -- § Extract pairwise binary data from a ternary distinction.
Dn3-to-binary : Distinction-n 3 → Dn3-binary-data
Dn3-to-binary d = record
 { S   = Distinction-n.S d
 ; p0 = Distinction-n.point d fz
 ; p1 = Distinction-n.point d (fs fz)
 ; p2 = Distinction-n.point d (fs (fs fz))
 ; sep01 = Distinction-n.distinct d fz (fs fz) λ ()
```

```
 ; sep02 = Distinction-n.distinct d fz (fs (fs fz)) λ ()
 ; sep12 = Distinction-n.distinct d (fs fz) (fs (fs fz)) λ ()
 ; cover = cover-3-nested d
 }

-- § Reconstruct a ternary distinction from pairwise binary data.
binary-to-Dn3 : Dn3-binary-data → Distinction-n 3
binary-to-Dn3 b = record
 { S        = S
 ; point    = pt
 ; distinct = dist
 ; cover    = cov
 }
 where
  open Dn3-binary-data b
  pt : Fin 3 → S
  pt fz           = p0
  pt (fs fz)      = p1
  pt (fs (fs fz)) = p2

  dist : (i j : Fin 3) → ¬ (i ≡ j) → ¬ (pt i ≡ pt j)
  dist fz          fz          h _ = h refl
  dist fz          (fs fz)     _ p = sep01 p
  dist fz          (fs (fs fz)) _ p = sep02 p
  dist (fs fz)     fz          _ p = sep01 (sym p)
  dist (fs fz)     (fs fz)     h _ = h refl
  dist (fs fz)     (fs (fs fz)) _ p = sep12 p
  dist (fs (fs fz)) fz         _ p = sep02 (sym p)
  dist (fs (fs fz)) (fs fz)    _ p = sep12 (sym p)
  dist (fs (fs fz)) (fs (fs fz)) h _ = h refl

  cov : (x : S) → Σ (Fin 3) (λ i → x ≡ pt i)
  cov x with cover x
  ... | inj₁ p          = fz , p
  ... | inj₂ (inj₁ p)   = fs fz , p
  ... | inj₂ (inj₂ p)   = fs (fs fz) , p

-- § Recoverable carrier and boundary fields for the ternary reconstruction.
record Dn3BinaryReconstruction : Set₁ where
 field
  extracted-carrier : (d : Distinction-n 3) →
   Dn3-binary-data.S (Dn3-to-binary d) ≡ Distinction-n.S d
  extracted-point-0 : (d : Distinction-n 3) →
   Dn3-binary-data.p0 (Dn3-to-binary d) ≡ Distinction-n.point d fz
  extracted-point-1 : (d : Distinction-n 3) →
   Dn3-binary-data.p1 (Dn3-to-binary d) ≡ Distinction-n.point d (fs fz)
  extracted-point-2 : (d : Distinction-n 3) →
   Dn3-binary-data.p2 (Dn3-to-binary d) ≡ Distinction-n.point d (fs (fs fz))
  reconstructed-carrier : (b : Dn3-binary-data) →
   Distinction-n.S (binary-to-Dn3 b) ≡ Dn3-binary-data.S b
  reconstructed-point-0 : (b : Dn3-binary-data) →
   Distinction-n.point (binary-to-Dn3 b) fz ≡ Dn3-binary-data.p0 b
  reconstructed-point-1 : (b : Dn3-binary-data) →
   Distinction-n.point (binary-to-Dn3 b) (fs fz) ≡ Dn3-binary-data.p1 b
  reconstructed-point-2 : (b : Dn3-binary-data) →
   Distinction-n.point (binary-to-Dn3 b) (fs (fs fz)) ≡ Dn3-binary-data.p2 b

 -- § The ternary record is recoverable from pairwise binary data at displayed fields.
theorem-Dn3-binary-reconstruction : Dn3BinaryReconstruction
theorem-Dn3-binary-reconstruction = record
 { extracted-carrier = λ _ → refl
 ; extracted-point-0 = λ _ → refl
 ; extracted-point-1 = λ _ → refl
 ; extracted-point-2 = λ _ → refl
 ; reconstructed-carrier = λ _ → refl
 ; reconstructed-point-0 = λ _ → refl
 ; reconstructed-point-1 = λ _ → refl
 ; reconstructed-point-2 = λ _ → refl
 }
```

What these proofs together establish is the formal counterpart of *Why binary, not unary or ternary* from Part I, in the precise sense available to the present framework. Singleton carriers

cannot host a distinction at all: the lemma distinction-n-1-collapses identifies every two elements of a unary carrier to coincide, and no-distinction-on-Dn1 closes the bridge by showing that no Distinction can inject into such a carrier without contradicting separated. The original record is exactly the $n = 2$ case of an n-ary template (distinction-to-Dn2, Dn2-to-distinction), with the carrier and boundary roundtrips bundled in theorem-distinction-Dn2-equivalence. At $n = 3$, the conversions Dn3-to-binary and binary-to-Dn3 prove more than that ternary data carries binary content: they prove that ternary data is recoverable from binary content, since the binary package of three pairwise separations and a nested binary cover reconstructs a Distinction-n 3. The recoverable carrier and boundary fields are collected in theorem-Dn3-binary-reconstruction.

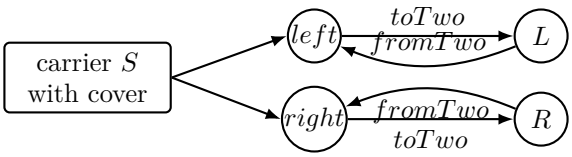
The scope of this last claim is precisely the scope of the framework. The theorem says that, inside this development, every inhabitant of Distinction-n 3 is recoverable from a binary package at the displayed carrier and boundary fields; it does not assert that all function fields are definitionally identical after a roundtrip, nor does it address alternative frameworks that take a ternary presentation as primitive. What is shown is that within the present record-based, `--safe --without-K` fragment of MLTT, the n-ary record at $n = 3$ adds no information beyond binary data plus a labelling. Binary is therefore the minimal cardinality at which the Distinction record is inhabitable in this framework, and higher arities reduce, in this framework, to binary structure plus a labelling map.

4 Normal Form

An isomorphism of distinctions preserves the two boundary roles and has inverse maps on carriers. The normal-form theorem says that every distinction is isomorphic, in this sense, to the canonical distinction on Two.

```
-- § Boundary-preserving isomorphism of distinction presentations.
record DistinctionIso (d e : Distinction) : Set₁ where
 private
  module D = Distinction d
  module E = Distinction e
 field
  to        : D.S → E.S
  from      : E.S → D.S
  to-left   : to D.left ≡ E.left
  to-right  : to D.right ≡ E.right
  from-left : from E.left ≡ D.left
  from-right : from E.right ≡ D.right
  to-from   : (y : E.S) → to (from y) ≡ y
  from-to   : (x : D.S) → from (to x) ≡ x
```

The map to Two is determined by the cover: an element is sent to L if it is the left boundary and to R if it is the right boundary.



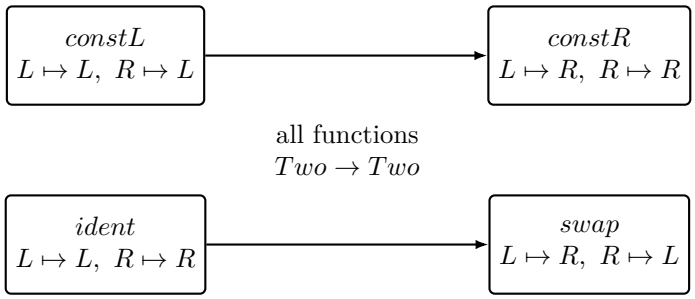

```
-- § The cover determines each carrier point's normal left/right code.
toTwo : (d : Distinction) → Distinction.S d → Two
toTwo d x with Distinction.cover d x
... | inj₁ _ = L
... | inj₂ _ = R
```

```
-- § The inverse candidate sends normal points back to the boundary points.
fromTwo : (d : Distinction) → Two → Distinction.S d
fromTwo d L = Distinction.left d
fromTwo d R = Distinction.right d

-- § Left boundary maps to L; the wrong branch contradicts separation.
toTwo-left : (d : Distinction) → toTwo d (Distinction.left d) ≡ L
toTwo-left d with Distinction.cover d (Distinction.left d)
... | inj₁ _ = refl
... | inj₂ p = ⊥-elim (Distinction.separated d p)

-- § Right boundary maps to R; the wrong branch contradicts separation.
toTwo-right : (d : Distinction) → toTwo d (Distinction.right d) ≡ R
toTwo-right d with Distinction.cover d (Distinction.right d)
... | inj₁ p = ⊥-elim (Distinction.separated d (sym p))
... | inj₂ _ = refl

-- § Roundtrip from Two back through the boundary map.
toTwo-fromTwo : (d : Distinction) → (y : Two) → toTwo d (fromTwo d y) ≡ y
toTwo-fromTwo d L = toTwo-left d
toTwo-fromTwo d R = toTwo-right d

-- § Roundtrip from an arbitrary carrier point through Two.
fromTwo-toTwo : (d : Distinction) → (x : Distinction.S d) → fromTwo d (toTwo d x) ≡ x
fromTwo-toTwo d x with Distinction.cover d x
... | inj₁ p = sym p
... | inj₂ p = sym p

-- § Normal form: every distinction is isomorphic to canonical Two.
two-normal-form : (d : Distinction) → DistinctionIso d Two-distinction
two-normal-form d = record
 { to        = toTwo d
 ; from      = fromTwo d
 ; to-left   = toTwo-left d
 ; to-right  = toTwo-right d
 ; from-left = refl
 ; from-right = refl
 ; to-from   = toTwo-fromTwo d
 ; from-to   = fromTwo-toTwo d
 }

-- § Equality discipline separating definitional and propositional equations.
record NormalFormEqualityDiscipline (d : Distinction) : Set where
 field
  fromTwo-L-definitional : fromTwo d L ≡ Distinction.left d
  fromTwo-R-definitional : fromTwo d R ≡ Distinction.right d
  toTwo-left-propositional : toTwo d (Distinction.left d) ≡ L
  toTwo-right-propositional : toTwo d (Distinction.right d) ≡ R
  toTwo-fromTwo-propositional : (y : Two) → toTwo d (fromTwo d y) ≡ y
  fromTwo-toTwo-propositional : (x : Distinction.S d) → fromTwo d (toTwo d x) ≡ x

-- § Normal-form equality discipline is witnessed by the roundtrip lemmas.
theorem-normal-form-equality-discipline :
 (d : Distinction) → NormalFormEqualityDiscipline d
theorem-normal-form-equality-discipline d = record
 { fromTwo-L-definitional = refl
 ; fromTwo-R-definitional = refl
 ; toTwo-left-propositional = toTwo-left d
 ; toTwo-right-propositional = toTwo-right d
 ; toTwo-fromTwo-propositional = toTwo-fromTwo d
 ; fromTwo-toTwo-propositional = fromTwo-toTwo d
 }
```

This is the whole normal-form argument. No choice is made in the map: the cover field gives only two possible cases, and the separation field rules out the contradictory boundary cases. The equality discipline is explicit. The map fromTwo computes by pattern matching, so its two endpoint equations close by refl. The equations involving toTwo are propositional equalities: they use the supplied cover and, when the wrong boundary is selected, the separation witness eliminates the impossible branch.

5 Endomorphisms of Two

An endomorphism of Two is fixed by its two values. There are therefore four cases: constant left, constant right, identity, and swap.




```
-- § The four possible endomorphism cases of Two.
data EndoCase : Set where
 constL : EndoCase
 constR : EndoCase
 ident : EndoCase
 swap : EndoCase

-- § Interpret a case label as an actual endofunction on Two.
interpret : EndoCase → Two → Two
interpret constL _ = L
interpret constR _ = R
interpret ident x = x
interpret swap L = R
interpret swap R = L

-- § Pointwise equality of endofunctions on Two.
Pointwise : (Two → Two) → (Two → Two) → Set
Pointwise f g = (x : Two) → f x ≡ g x
```

The classifier simply inspects the two values of a function.

```
-- § Classify an endofunction by its values at L and R.
classify : (Two → Two) → EndoCase
classify f with f L | f R
... | L | L = constL
... | R | R = constR
... | L | R = ident
... | R | L = swap

-- § Soundness: interpretation of the classified case recovers the function pointwise.
classify-sound : (f : Two → Two) → Pointwise f (interpret (classify f))
classify-sound f L with f L | f R
... | L | L = refl
... | R | R = refl
... | L | R = refl
... | R | L = refl
classify-sound f R with f L | f R
... | L | L = refl
... | R | R = refl
... | L | R = refl
... | R | L = refl
```

Uniqueness follows by evaluating at L and R. Any attempted identification of two different cases identifies either L with R, or conversely.

```
-- § Distinct case labels cannot have the same interpreted behavior.
case-unique : (c d : EndoCase) → Pointwise (interpret c) (interpret d) → c ≡ d
case-unique constL constL _ = refl
```

```
case-unique constL constR p = ⊥-elim (L≠R (p L))
case-unique constL ident  p = ⊥-elim (L≠R (p R))
case-unique constL swap   p = ⊥-elim (L≠R (p L))
case-unique constR constL p = ⊥-elim (R≠L (p L))
case-unique constR constR _ = refl
case-unique constR ident  p = ⊥-elim (R≠L (p L))
case-unique constR swap   p = ⊥-elim (R≠L (p R))
case-unique ident  constL p = ⊥-elim (R≠L (p R))
case-unique ident  constR p = ⊥-elim (L≠R (p L))
case-unique ident  ident  _ = refl
case-unique ident  swap   p = ⊥-elim (L≠R (p L))
case-unique swap constL p = ⊥-elim (R≠L (p L))
case-unique swap constR p = ⊥-elim (L≠R (p R))
case-unique swap ident  p = ⊥-elim (R≠L (p L))
case-unique swap swap   _ = refl

-- § Classification is unique among all pointwise-correct case labels.
classify-unique :
 (f : Two → Two) → (c : EndoCase) → Pointwise f (interpret c) → classify f ≡ c
classify-unique f c p =
 case-unique (classify f) c
  (λ x → trans (sym (classify-sound f x)) (p x))
```

6 Why the Closure is K_4

Composition of endomorphisms is total: for any two endomorphism cases, their composite is again an endomorphism of Two, hence is one of the four cases just classified.

```
-- § Composition of endofunctions on Two.
_∘₂_ : (Two → Two) → (Two → Two) → Two → Two
(f ∘₂ g) x = f (g x)

-- § Compose two case labels by classifying the composite function.
composeCase : EndoCase → EndoCase → EndoCase
composeCase c d = classify (interpret c ∘₂ interpret d)

-- § Endomorphisms are closed under composition.
endomorphisms-closed :
 (c d : EndoCase) → Σ EndoCase (λ e → Pointwise (interpret e) (interpret c ∘₂ interpret d))
endomorphisms-closed c d =
 composeCase c d , λ x → sym (classify-sound (interpret c ∘₂ interpret d) x)
```

The closure is introduced only at this point. It packages the four classified endomorphism cases under an explicit representation contract. Unpacking that contract yields the following three conditions.

1. *Separation.* Distinct endomorphism cases must remain distinct on the carrier. No identification of cases is allowed.

2. *Realisation.* Every endomorphism case must be represented by a vertex of the carrier.

3. *No surplus.* Every vertex of the carrier must arise from one of these cases.

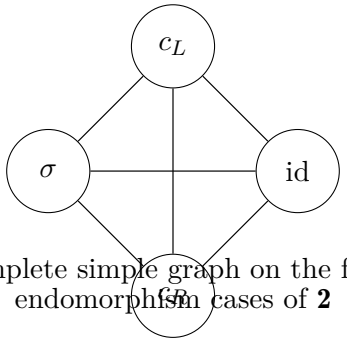
These conditions make explicit what "representation" means here: each endomorphism case is carried as a distinct and retrievable element of the carrier, and every element of the carrier corresponds to such a case. Without separation, cases collapse; without realisation, cases are missing; without the no-surplus condition, additional structure is introduced. The contract is stated before it is used.

Under these three conditions, a carrier with fewer than four represented vertices cannot represent the four cases without identification. If two cases are identified, case-unique shows that the

corresponding functions agree pointwise; but the only way different rows of the four case table agree is by identifying L and R, contradicting L≠R. Thus separation and realisation require four represented vertices.

An additional vertex is not licensed by the endomorphism data. The composite of any two represented cases is already one of the four classified cases, by endomorphisms-closed. A new vertex would not be the value of any composition demanded by the case table; it would violate no surplus.

Finally, total composability is not another assumption. Any ordered pair of endomorphisms has a composite, so there is no missing composability relation between distinct represented cases. The underlying simple graph is therefore the complete graph on four vertices, K_4.



complete simple graph on the four endomorphism cases of **2**

The following small records package the three conditions. The record FaithfulClosure contains realisation and separation: realize supplies a vertex for each case, and reflect-realize prevents two cases from being identified. The record MinimalClosure adds no surplus through no-extra.

These conditions are therefore not independent choices once this notion of representation has been fixed; they unpack that notion.

```
-- § Representation with realisation, reflection, and separation of cases.
record FaithfulClosure : Set₁ where
 field
  V       : Set
  realize : EndoCase → V
  reflect : V → EndoCase
  reflect-realize : (c : EndoCase) → reflect (realize c) ≡ c

-- § Minimal closure adds no surplus vertices beyond represented cases.
record MinimalClosure : Set₁ where
 field
  closure : FaithfulClosure
  no-extra : (v : FaithfulClosure.V closure) →
             Σ EndoCase (λ c → FaithfulClosure.realize closure c ≡ v)

 -- § The canonical vertex carrier is the four-case type itself.
K4Vertex : Set
K4Vertex = EndoCase

 -- § Edges connect distinct cases in the induced simple graph.
K4Edge : K4Vertex → K4Vertex → Set
K4Edge c d = ¬ (c ≡ d)

 -- § Canonical minimal closure of the four endomorphism cases.
canonicalK4 : MinimalClosure
canonicalK4 = record
 { closure = record
    { V       = EndoCase
    ; realize = λ c → c
    ; reflect = λ c → c
    ; reflect-realize = λ c → refl
```

```
  }
; no-extra = λ v → v , refl
}

-- § No-surplus gives the other roundtrip: realize after reflect returns the vertex.
realize-reflect :
(m : MinimalClosure) →
(v : FaithfulClosure.V (MinimalClosure.closure m)) →
FaithfulClosure.realize (MinimalClosure.closure m)
 (FaithfulClosure.reflect (MinimalClosure.closure m) v)
≡ v
realize-reflect m v with MinimalClosure.no-extra m v
... | c , p = trans (sym (cong realize q)) p
 where
 fc = MinimalClosure.closure m
 realize = FaithfulClosure.realize fc
 reflect = FaithfulClosure.reflect fc
 reflect-realize = FaithfulClosure.reflect-realize fc

 q : c ≡ reflect v
 q = trans (sym (reflect-realize c)) (cong reflect p)
```

The same information can be displayed as a compact K_4 presentation. This is not the large invariant record of the main kernel; it is the companion-level graph layer induced by any minimal representation of the four endomorphism cases. Vertices are the carrier of the closure, edges connect vertices whose reflected cases are distinct, and the two roundtrips certify that no case is lost and no vertex is surplus.

```
-- § Compact graph-level presentation of the minimal four-case closure.
record K4Presentation : Set₁ where
 field
  vertices              : Set
  edge                  : vertices → vertices → Set
  realize-vertex        : EndoCase → vertices
  reflect-vertex        : vertices → EndoCase
  reflect-realize-vertex : (c : EndoCase) → reflect-vertex (realize-vertex c) ≡ c
  realize-reflect-vertex : (v : vertices) → realize-vertex (reflect-vertex v) ≡ v
  edge-complete         : (v w : vertices) → edge v w ≡ (¬ (reflect-vertex v ≡ reflect-vertex w))
  represented-closure : MinimalClosure

 -- § Any minimal closure yields a K4 presentation.
minimal-closure-presentation : (m : MinimalClosure) → K4Presentation
minimal-closure-presentation m = record
 { vertices              = FaithfulClosure.V fc
 ; edge                  = λ v w → ¬ (FaithfulClosure.reflect fc v ≡ FaithfulClosure.reflect fc w)
 ; realize-vertex        = FaithfulClosure.realize fc
 ; reflect-vertex        = FaithfulClosure.reflect fc
 ; reflect-realize-vertex = FaithfulClosure.reflect-realize fc
 ; realize-reflect-vertex = realize-reflect m
 ; edge-complete         = λ _ _ → refl
 ; represented-closure = m
 }
 where
  fc = MinimalClosure.closure m

-- § The canonical K4 presentation is induced by canonicalK4.
canonicalK4Presentation : K4Presentation
canonicalK4Presentation = minimal-closure-presentation canonicalK4
```

The three record fields are not three independent stipulations. Each one is exactly the load-bearing piece of one of the three representation conditions, and the following lemmas name that load. The proofs are short because the conditions are exactly the bijection data; that is the point.

Separation is load-bearing. The field reflect-realize makes realize injective. If two cases were identified on the carrier, applying reflect to the identification would identify the cases themselves, contradicting the four-case enumeration of EndoCase.

```
-- § Separation is load-bearing: realize is injective.
realize-injective :
 (m : MinimalClosure) →
 (c d : EndoCase) →
 FaithfulClosure.realize (MinimalClosure.closure m) c
  ≡ FaithfulClosure.realize (MinimalClosure.closure m) d →
 c ≡ d
realize-injective m c d p =
 trans (sym (FaithfulClosure.reflect-realize (MinimalClosure.closure m) c))
  (trans (cong (FaithfulClosure.reflect (MinimalClosure.closure m)) p)
   (FaithfulClosure.reflect-realize (MinimalClosure.closure m) d))
```

Realisation is load-bearing. The totality of realize on EndoCase is exactly the demand that every endomorphism case is hosted by some vertex. Without it, some case has no representative and the carrier fails to support the composition table at all.

```
-- § Realisation is load-bearing: every case has a vertex.
case-has-vertex :
 (m : MinimalClosure) →
 (c : EndoCase) →
 Σ (FaithfulClosure.V (MinimalClosure.closure m))
    (λ v → FaithfulClosure.realize (MinimalClosure.closure m) c ≡ v)
case-has-vertex m c =
 FaithfulClosure.realize (MinimalClosure.closure m) c , refl
```

No surplus is load-bearing. The field no-extra forbids vertices that do not arise from any endomorphism case. Without it, the carrier may host data that is not licensed by the endomorphism set, breaking the claim that the closure adds nothing to its datum.

```
-- § No surplus is load-bearing: every vertex comes from a case.
vertex-from-case :
 (m : MinimalClosure) →
 (v : FaithfulClosure.V (MinimalClosure.closure m)) →
 Σ EndoCase
    (λ c → FaithfulClosure.realize (MinimalClosure.closure m) c ≡ v)
vertex-from-case m v = MinimalClosure.no-extra m v
```

Dropping reflect-realize re-admits identification of cases. Dropping the totality of realize re-admits cases without vertices. Dropping no-extra re-admits vertices without cases. In each case, the remaining structure no longer represents the four-case set in the intended sense. The three fields are therefore not chosen because they yield K_4; within this contract they are fixed because dropping any one breaks representation, and K_4 follows.

The companion proves these three load-bearing lemmas against its own slim records FaithfulClosure and MinimalClosure so that the file remains self-contained and type-checks without the larger source. The full development `Void.lagda.tex` carries the analogous statements against the richer K4Record used there; the present file does not import them, it reproduces them at the level of detail that this note needs.

This is the sense in which the closure yields K_4: the represented vertex set is the four-case set, and the induced simple graph connects every distinct pair. The equations reflect-realize and realize-reflect show that any carrier satisfying the three conditions is equivalent to the four-case carrier.

Equivalently, for any m : MinimalClosure, the carrier

$$\mathsf{FaithfulClosure.V}\,(\mathsf{MinimalClosure.closure}\,m)$$

is in bijection with EndoCase via realize and reflect. The field reflect-realize proves one composite to be the identity on EndoCase; realize-reflect proves the other composite to be the identity on the carrier. If composition on the carrier is read through this bijection,

$$v \star_m w = \mathsf{realize}_m(\mathsf{composeCase}\,(\mathsf{reflect}_m\,v)(\mathsf{reflect}_m\,w)),$$

then reflecting the composite recovers the four-case composition table of EndoCase. In this precise sense every minimal closure satisfying separation, realisation, and no surplus is isomorphic to the canonical four-case carrier. Thus K_4 is not only minimal; it is unique up to isomorphism under the stated constraints.

7 Identity of indiscernibles, executed

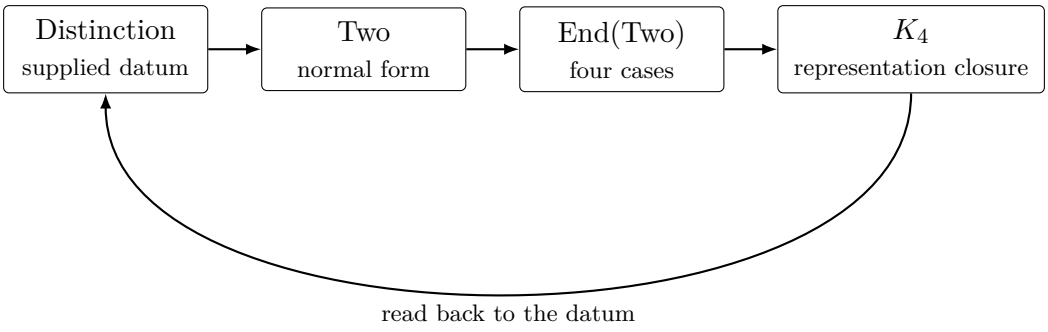
The earlier section on why two cannot occur without separation invoked the pressure of the identity of indiscernibles: if two alleged positions cannot be distinguished by anything expressible in the system, the system has no internal ground for holding them apart. This section does not introduce that pressure for the first time. It records only what becomes possible after a concrete carrier and equality discipline have already been chosen: MLTT/Agda executes one precise instance of the principle. The induction principle for propositional equality yields it directly. If every predicate true of a transfers to b, choose the predicate $P\,x = a \equiv x$. Since a equals itself by refl, the transfer gives $a \equiv b$.

```
-- § Identity of indiscernibles: indistinguishable elements are propositionally equal.
indiscernibility-implies-equality :
 {S : Set} (a b : S) →
 ((P : S → Set) → P a → P b) →
  a ≡ b
indiscernibility-implies-equality a b ind = ind (λ x → a ≡ x) refl
```

The Agda term is one realisation of the principle, not its source. Agda does not create the thought that indiscernibles collapse; it checks this particular execution of it. Inside the present setting, the role of Distinction is now clear: the separated field prevents the two boundary points from collapsing into one, and the cover field prevents a third, unaccounted position from being hidden in the carrier.

8 Re-projection to the datum

The K_4 closure does not replace the original distinction; it presupposes it. This section makes the dependency explicit at two levels of strength so that no reader can mistake the closure for an independent foundation, and so that the trivial direction is not advertised as more than it is.



The structural content as a checked term. The bijection between the carrier of any minimal closure and the four endomorphism cases is not left to commentary. It is bundled as a record and constructed from the lemmas already proved: reflect-realize is one round-trip, realize-reflect is the other. This is the load-bearing structural map.

```
-- § Bijection record: two maps and two inverse laws.
record Bijection (A B : Set) : Set where
 field
  to      : A → B
  from    : B → A
  to-from : (b : B) → to (from b) ≡ b
  from-to : (a : A) → from (to a) ≡ a

-- § Every minimal closure carrier is in bijection with EndoCase.
closure-bijection-EndoCase :
 (m : MinimalClosure) →
 Bijection (FaithfulClosure.V (MinimalClosure.closure m)) EndoCase
closure-bijection-EndoCase m = record
 { to     = FaithfulClosure.reflect  (MinimalClosure.closure m)
 ; from   = FaithfulClosure.realize (MinimalClosure.closure m)
 ; to-from = FaithfulClosure.reflect-realize (MinimalClosure.closure m)
 ; from-to = realize-reflect m
 }
```

This term is the actual dependency. Every minimal closure has a carrier in checked bijection with EndoCase; by two-normal-form, EndoCase is built from Two, the canonical carrier of any binary distinction. The four-vertex structure is therefore not an independent foundation; it is, up to the bijection just constructed, the endomorphism set of the normal form of any binary distinction.

Canonical re-projection. At the level of returning a value of type Distinction, every minimal closure re-projects to the canonical binary distinction. This is not a carrier-level isomorphism between four vertices and two points; such an isomorphism would be false. The carrier-level bijection is the checked map to EndoCase above. The re-projection below records the dependency at the Distinction level: the four-case closure is readable only through the binary normal form whose endomorphisms it represents.

```
-- § Reproject a minimal closure to the canonical distinction it presupposes.
k4-presupposes-distinction : MinimalClosure → Distinction
k4-presupposes-distinction _ = Two-distinction

-- § Full checked chain from a distinction to normal form, K4 closure, and return.
record FormalClosureChain (d : Distinction) : Set₁ where
 field
  binary-normal-form        : DistinctionIso d Two-distinction
  equality-discipline       : NormalFormEqualityDiscipline d
  endomorphism-classifier   : (Two → Two) → EndoCase
  classifier-is-sound       : (f : Two → Two) →
   Pointwise f (interpret (endomorphism-classifier f))
  classifier-is-unique      :
   (f : Two → Two) (c : EndoCase) →
   Pointwise f (interpret c) → endomorphism-classifier f ≡ c
  composition-is-closed     :
   (c e : EndoCase) →
   Σ EndoCase (λ r → Pointwise (interpret r) (interpret c ∘₂ interpret e))
  k4-presentation           : K4Presentation
  closure-carrier-bijection : Bijection (K4Presentation.vertices k4-presentation) EndoCase
  reprojected-distinction   : Distinction
  reprojection-is-canonical : reprojected-distinction ≡ Two-distinction

-- § The formal closure chain is witnessed for every binary distinction.
theorem-formal-closure-chain :
 (d : Distinction) → FormalClosureChain d
theorem-formal-closure-chain d = record
 { binary-normal-form        = two-normal-form d
 ; equality-discipline       = theorem-normal-form-equality-discipline d
 ; endomorphism-classifier   = classify
 ; classifier-is-sound       = classify-sound
 ; classifier-is-unique      = classify-unique
 ; composition-is-closed     = endomorphisms-closed
 ; k4-presentation           = canonicalK4Presentation
 ; closure-carrier-bijection = closure-bijection-EndoCase canonicalK4
 ; reprojected-distinction   = k4-presupposes-distinction canonicalK4
```

```
 ; reprojection-is-canonical = refl
 }

-- § Same closure chain, starting from the n=2 distinction template.
record Dn2FormalClosureChain (d : Distinction-n 2) : Set₁ where
 field
  n2-equivalence : DistinctionDn2Equivalence
  left-is-point-0    : Distinction.left (Dn2-to-distinction d) ≡ Distinction-n.point d fz
  right-is-point-1   : Distinction.right (Dn2-to-distinction d) ≡ Distinction-n.point d (fs fz)
  closure-chain    : FormalClosureChain (Dn2-to-distinction d)

-- § The n=2 template closes through the same normal-form and K4 route.
theorem-Dn2-formal-closure-chain :
 (d : Distinction-n 2) → Dn2FormalClosureChain d
theorem-Dn2-formal-closure-chain d = record
 { n2-equivalence = theorem-distinction-Dn2-equivalence
 ; left-is-point-0    = refl
 ; right-is-point-1   = refl
 ; closure-chain    = theorem-formal-closure-chain (Dn2-to-distinction d)
 }
```

The full formalisation uses a richer K4Record. The structural dependency carries over without change: the closure carrier is not an independent foundation, it is, by closure-bijection-EndoCase, in checked bijection with the endomorphism set of the binary normal form. The record theorem-formal-closure-chain bundles the route as one object, and theorem-Dn2-formal-closure-chain states the same route from the $n = 2$ template.

Part III. Frame

9 Scope

The formal result is conditional. The development never constructs an inhabitant of Distinction without supplied data; every Distinction-valued construction below the abstract takes a Distinction as input or returns the canonical Two-distinction. The code therefore establishes the consequences of the threshold datum, not the existence of that datum outside the formal setting.

The conceptual part identifies the dependency that motivates the datum: relations, morphisms, judgements, and equations use distinguishable positions before they can be written down. This is a claim about the conditions under which formalisation becomes available. The checked part begins only after the data are granted.

The machine-checked result is the conditional consequence. Given a structure with two distinguishable, exhaustive points, the normal form is Two; given the endomorphisms of that normal form, representing the four cases with separation, realisation, and no surplus yields the four-case carrier with complete simple adjacency; and the closure returns to the datum from which it was built.

The theory of formulability names the architectural reading of this chain. It treats the local question answered here–what licenses a system to keep two points apart–as the first inspectable threshold of formal description. The checked kernel explains what follows from the stated datum; the larger `Void.lagda.tex` development asks how far the required infrastructure can be rebuilt internally.

10 Placement

This section locates the contribution so that the formal result is read at the right level.

Leibniz. The pressure behind the entry question is not invented by the companion. It is the thought behind the identity of indiscernibles: if no expressible difference separates two alleged positions, the system has no internal ground for holding them apart. The Agda term indiscernibility-implies-equality is one execution of this pressure after a carrier has been chosen, not the source of the pressure itself.

Lawvere. "Adjointness in Foundations" (1969) treats truth and falsity as adjoint to the act of distinguishing. The present note operates one level below: before truth-values are assigned, before propositions are evaluated. Two is a fresh carrier, deliberately unread as $\{\bot, \top\}$, so that the closure result depends on separation alone and not on a prior reading of the two points as truth-values.

Univalent foundations. Two is an h-set, and the development uses `--without-K`. No higher-dimensional structure is assumed or invoked. The note is therefore compatible with both intensional Martin-Löf type theory and a univalent reading; nothing in the closure result depends on the choice of identity discipline.

The contribution is a machine-checked conditional statement at a precise threshold: given explicit separation and cover, the normal form is Two; its endomorphisms have four cases; under the stated representation conditions the corresponding closure is K_4, unique up to isomorphism; and that closure points back to the data from which it was built.

11 Conclusion and Outlook

The opening question was: a formal system says that there are two; what inside the system licenses that claim? The answer delivered by the checked part of this note is deliberately conditional. Once a carrier, two boundary points, a proof of non-coincidence, and an exhaustive cover are supplied, the datum has canonical two-point normal form. Its endomorphisms classify into exactly four cases. Under separation, complete realisation, and no surplus, the representation of those four cases is the K_4 closure. That closure does not replace the original datum; it points back to it.

This closes the companion's formal question. The note shows what follows when the threshold data are made explicit. The result is a small closed argument: the beginning asks what licenses two; the end answers that two is licensed, in this formal fragment, by separation against collapse and cover against surplus.

The outlook therefore points to `Void.lagda.tex`, not to material outside the formal build. The larger file continues the same discipline: keep the formal infrastructure inside the development as far as Agda permits. Apart from the primitive universe machinery needed by Agda, the later construction does not import ready-made equality, arithmetic, rationals, or real analysis as background. It rebuilds the route from natural numbers to integers, from integers to rationals, and from rational distance and order toward Cauchy-style real structure.

In that sense the next question is not what the closure means outside the formal development. The next question is how much of the machinery usually taken for granted can be reconstructed after the binary threshold has been fixed. The companion isolates the first closed kernel; `Void.lagda.tex` tests how far that kernel can carry a self-contained formal build.

12 Navigation into Void

The corresponding self-contained development is in the repository de-johannes/Void-and-Form. The main file is `Void.lagda.tex`. The route through the larger formal development is:

- *Void as boundary*: *What Void Does Not Name*, *Why Formal Systems Cannot Begin Themselves*, and *Representation Requires A Cut*.

- *The binary threshold*: *Why The First Cut Is Binary*, *The Formal Threshold*, and the record `Distinction`.
- *Normal form and classification*: *Elimination and Classification*, *Two Normal Form*, and the endomorphism classification of `Two -> Two`.
- *Four-vertex closure*: *Graph Presentation*, *The K4 Invariant Record*, *K4 Representation Theory*, and `K4Record-is-canonical`.
- *Return to the datum*: `record-presupposes-distinction`, the formal witness that the surviving closure entails the originating distinction.
- *Internal arithmetic*: the construction of natural numbers, integer normal forms, rational numbers, rational equality, order, and distance inside the file rather than by standard-library import.
- *Cauchy route*: the rational setoid and order laws, triangle inequality infrastructure, and Cauchy-style convergence material that begin the passage from rational numbers toward real structure.